

“Design and Development of a Generative AI Assistant and Knowledge Base for Pharmaceuticals”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

ANTHONY NISSY : 2520030064

P. ANURADHA NANDINI : 2520030171

K. SHASHANK REDDY : 2520030215

Under the Esteemed Guidance of

Dr . S . Spandana

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “Design and Development of a Generative AI Assistant and Knowledge Base for Pharmaceuticals” is a record of Bonafide work of ANTHONY NISSY 2520030064, P. ANURADHA NANDINI – 2520030171 , SHASHANK– 2520030215 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

ANTHONY NISSY – 2520030064
P. ANURADHA NANDINI – 2520030171
K.SHASHANK REDDY– 2520030215

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “Design and Development of a Generative AI Assistant and Knowledge Base for Pharmaceuticals” is a bonafide work done and submitted by ANTHONY NISSY – 2520030064, P. ANURADHA NANDINI 2520030171, K. SHASHANK REDDY– 2520030215 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator Dr. R .Satish Kumar and our guide Dr. S.Spandana Department of Computer Science, for the tremendous support, encouragement and guidance that cannot be expressed in words.

We express our gratitude to Dr. P. Venkateshwara Rao (CSE) Head of the Department for Computer Science Engineering for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

ANTHONY NISSY – 2520030064
P. ANURADHA NANDINI – 2520030171
K. SHASHANK REDDY -2520030215

Abstract

This project presents the design and development of a Generative AI Assistant and Knowledge Base for Pharmaceuticals, a full-stack, microservices-based system that combines large language model (LLM) capabilities with structured and unstructured pharmaceutical data to deliver accurate, context-aware responses to queries related to drug information, interactions, dosage guidelines, and regulatory documentation.

The system employs a polyglot persistence architecture: a normalised relational database (PostgreSQL) stores structured entities such as drugs, manufacturers, prescriptions, and user records with optimised SQL queries and transactional integrity; a NoSQL document store (MongoDB) manages semi-structured data such as drug leaflets, clinical notes, and FAQs; and a vector database (pgvector) enables semantic search and retrieval-augmented generation (RAG) by storing embeddings of pharmaceutical literature, allowing the AI assistant to retrieve contextually relevant knowledge before generating responses.

The backend is architected as a set of independently deployable microservices, built using FastAPI (with Pydantic validation, JWT-based authentication, dependency injection, and OpenAPI documentation) for the AI/RAG and query-processing services, and Node.js/Express or Spring Boot for supporting services such as user management, catalog management, and notification handling. Services communicate via REST APIs and asynchronous messaging (Kafka), decoupling the system into bounded contexts such as authentication, knowledge retrieval, chat/inference, and admin management, evolving from an initial monolithic prototype into a microservices architecture through systematic decomposition.

The complete system is containerised using Docker, orchestrated for local and cloud deployment, and supported by a CI/CD pipeline for automated testing and deployment. System design is documented using the C4 model to capture context, container, component, and code-level views, ensuring maintainability and clarity for future development.

The final deliverable is a live, demonstrable full-stack application where a frontend interface connects to all backend microservices, allowing pharmacists, clinicians, or patients to query a generative AI assistant that grounds its answers in a verified, continuously updatable pharmaceutical knowledge base — reducing the risk of hallucinated medical information while improving accessibility to reliable drug-related knowledge.

Index

S. No	Chapters	Topics	Page No.
		Team Details	8
		Frameworks / Platforms / Software Used	8
		Acknowledgement, Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	9
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	12
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version Control 3.5 APIs or External services	13
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture (CI/CD)	15
5	Design	5.1 C4 model views 5.2 ER Diagram / Database schema	17
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & Authorization	18
7	Features	7.1 List of main features 7.2 How each feature works	20
8	Testing	8.1 Postman testing 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	21
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	23
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	24
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	25
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	26
13	References	Books, tutorials, APIs, documentation sites used	27
14	Appendices	Screenshots, sample code, installation/setup instructions, user manual, geo-tag photos, review forms	28

Team Details

Roll Number(s)	Name of the Student	Signature
2520030064	Anthony Nissy	
2520030171	P. Anuradha Nandini	
2520030215	K. Shashank Reddy	

Frameworks / Platforms / Software Used

Software / Technologies Required:

Category	Technology
Operating System	Windows 10/11, Linux, or macOS
Frontend	HTML5, CSS3, JavaScript / React.js
Backend	Python with FastAPI
Additional Backend	Node.js with Express.js / Spring Boot
Databases	PostgreSQL and MongoDB
Vector Database	PostgreSQL with pgvector
Generative AI	LLM / Generative AI API
RAG	Embedding model and Retrieval-Augmented Generation framework
API Testing	Postman
Messaging	Apache Kafka
Containerization	Docker and Docker Compose
Version Control	Git and GitHub
CI/CD	GitHub Actions
Documentation	OpenAPI / Swagger and C4 Model
Development Environment	Visual Studio Code
Web Browser	Google Chrome / Microsoft Edge / Mozilla Firefox

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Access to accurate drug information is central to safe and effective healthcare. Pharmacists, clinicians and patients routinely need to know the indications, dosage guidelines, contraindications, side effects and possible interactions of a medicine. This information is spread across product leaflets, clinical notes, drug databases, formularies and regulatory documents, most of which are long, technical and published in different formats.

Recent advances in Generative AI and Large Language Models (LLMs) make it possible to answer such questions in natural language. However, a general-purpose LLM used on its own can hallucinate: it may produce fluent but incorrect statements about dosage or interactions, which is unacceptable in a medical setting. It also has no access to the latest regulatory updates or to an organisation's own documents.

Retrieval-Augmented Generation (RAG) addresses this problem. Instead of relying only on the model's memory, the system first retrieves relevant passages from a verified knowledge base and then asks the LLM to compose an answer grounded in those passages. This project applies RAG to the pharmaceutical domain and combines it with modern software-engineering practice: a microservices architecture, polyglot persistence, event-driven messaging, containerisation and CI/CD.

The project builds on several core areas of computer science:

- Generative AI and NLP: embeddings, semantic search, prompt design and RAG pipelines.
- Database Management: relational modelling (PostgreSQL), document stores (MongoDB) and vector search (pgvector).
- Distributed Systems: microservices, REST APIs, asynchronous messaging with Apache Kafka.
- DevOps: Docker, Docker Compose, GitHub Actions for automated build, test and deployment.
- Software Architecture: documentation using the C4 model and OpenAPI/Swagger.

The system began as a monolithic prototype and was systematically decomposed into independently deployable services organised around bounded contexts, which improves scalability, fault isolation and maintainability.

1.2 Problem Statement

Drug-related knowledge is fragmented across structured databases, semi-structured leaflets and clinical notes, and unstructured literature. Users must search several sources manually, which is slow and error-prone. Existing chatbots that rely solely on an LLM can generate unverified or fabricated medical content, and they cannot cite the source of an answer.

The core problem addressed by this project is the absence of an integrated, trustworthy and continuously updatable system that (i) unifies structured, semi-structured and unstructured pharmaceutical data, (ii) answers natural-language queries about drugs, interactions, dosage and regulatory documents, and (iii) grounds every answer in verified sources to reduce hallucination.

1.3 Scope of the Project

In scope:

- A knowledge base of drugs, manufacturers, prescriptions, interactions, leaflets, FAQs and clinical notes stored using PostgreSQL, MongoDB and pgvector.
- A RAG-based generative AI assistant that answers pharmaceutical queries with source references.
- Microservices for authentication/user management, catalog management, AI/RAG, chat/query processing and notifications, communicating through REST and Kafka.
- A web frontend for pharmacists, clinicians, patients and administrators.
- Containerisation with Docker, a GitHub Actions CI/CD pipeline, OpenAPI documentation and C4 architecture diagrams.

Out of scope:

- The assistant is an information-support tool. It does not diagnose conditions, prescribe treatment or replace professional medical judgement.
- Integration with hospital information systems and real patient health records is not included in this version.

1.4 Objectives of the Project

1. To design a polyglot persistence layer combining PostgreSQL, MongoDB and pgvector for pharmaceutical data.
2. To implement a RAG pipeline that retrieves relevant knowledge before generating a response, and returns the sources used.
3. To build independently deployable microservices using FastAPI and Node.js/Express or Spring Boot with JWT-based security.
4. To decouple services using REST APIs and asynchronous Kafka messaging.
5. To containerise the complete system with Docker and automate testing and deployment with CI/CD.
6. To document the design using the C4 model and OpenAPI/Swagger.
7. To deliver a live, demonstrable full-stack application connecting the frontend to all backend services.

1.5 Importance of the Application

- Patient safety: grounding answers in verified sources reduces the risk of hallucinated medical information.
- Time saving: pharmacists and clinicians obtain consolidated answers in seconds instead of searching several documents.
- Accessibility: patients can understand medicines through plain-language explanations.
- Updatability: new leaflets and guidelines can be added to the knowledge base without retraining the model.
- Maintainability: the microservices and C4 documentation allow teams to evolve each component independently.

1.6 Target Users / Audience

User	How they use the system
Pharmacists	Verify dosage, check interactions, look up alternatives and manufacturer details.
Clinicians	Quickly retrieve drug information, guidelines and regulatory notes to support prescribing decisions.
Patients	Ask simple questions about their medicines, usage instructions and side effects.
Administrators	Manage users, upload and update knowledge documents, monitor the system.

CHAPTER - 2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i5 / Ryzen 5 (4 cores)	Intel Core i7 / Ryzen 7 (8 cores)
RAM	8 GB	16 GB or more
Storage	20 GB free (SSD)	50 GB free (SSD)
Network	Broadband internet (for LLM API)	Stable high-speed connection
Display	1366 × 768	1920 × 1080

Because the LLM and embedding models are accessed through APIs, a dedicated GPU is not required. Running several containers (PostgreSQL, MongoDB, Kafka and the services) simultaneously is the main reason for the recommended 16 GB of RAM.

2.2 Software Requirements

Category	Requirement
Operating System	Windows 10/11, Linux or macOS
Languages / Runtimes	Python 3.10+, Node.js (LTS), Java (if Spring Boot is used)
Databases	PostgreSQL with pgvector extension; MongoDB
Messaging	Apache Kafka
Containers	Docker and Docker Compose
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox
API Testing	Postman

2.3 Development Tools and Frameworks

- Visual Studio Code as the primary development environment.
- Git and GitHub for version control and collaboration.
- FastAPI, Pydantic and Uvicorn for Python services.
- Node.js with Express.js (or Spring Boot) for supporting services.
- React.js with HTML5, CSS3 and JavaScript for the frontend.
- Postman for API testing; Swagger UI / OpenAPI for interactive API documentation.
- GitHub Actions for CI/CD; Docker Compose for local orchestration.

CHAPTER - 3 TECHNOLOGY STACK

3.1 Front-End Design

The user interface is built with HTML5, CSS3 and JavaScript using React.js. React's component model suits the application: a chat window, a message list, a source-citation panel, a drug search page and an admin dashboard are each implemented as reusable components. State (logged-in user, chat history, loading status) is managed with React hooks and context. CSS is used to create a responsive layout that works on desktops, tablets and phones. The frontend communicates with the backend only through REST APIs exposed by the API gateway.

3.2 Back-End Design

The backend is a set of independently deployable microservices:

- **FastAPI (Python):** used for the AI/RAG service and the chat/query-processing service. FastAPI provides Pydantic request validation, dependency injection, JWT authentication support, asynchronous request handling and automatic OpenAPI documentation.
- **Node.js / Express.js or Spring Boot:** used for supporting services – user management, catalog management and notification handling.
- **Apache Kafka:** provides asynchronous, event-driven communication between services (for example, “document uploaded” events that trigger embedding generation, and “alert” events that trigger notifications).

3.3 Database Design

The system uses polyglot persistence – each type of data is stored in the engine best suited to it.

Store	Data	Reason
PostgreSQL	Drugs, manufacturers, interactions, prescriptions, users, chat sessions	Normalised schema, transactional integrity, optimised SQL queries and joins
MongoDB	Drug leaflets, clinical notes, FAQs	Flexible, semi-structured documents with varying fields
pgvector (PostgreSQL)	Embeddings of pharmaceutical literature chunks	Similarity search for semantic retrieval in the RAG pipeline

3.4 Version Control

Git is used for version control and the project is hosted on GitHub. A feature-branch workflow is followed: each feature or service change is developed on its own branch and merged through a pull request after the CI pipeline passes. Commit history documents the evolution from the initial monolith to the microservices architecture.

3.5 APIs or External Services

- LLM / Generative AI API – generates the natural-language answer from the retrieved context.
- Embedding model API – converts document chunks and user queries into vectors.
- Internal REST APIs – documented with OpenAPI/Swagger and tested with Postman.
- GitHub Actions – external CI/CD service used for automated testing and deployment.

CHAPTER - 4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

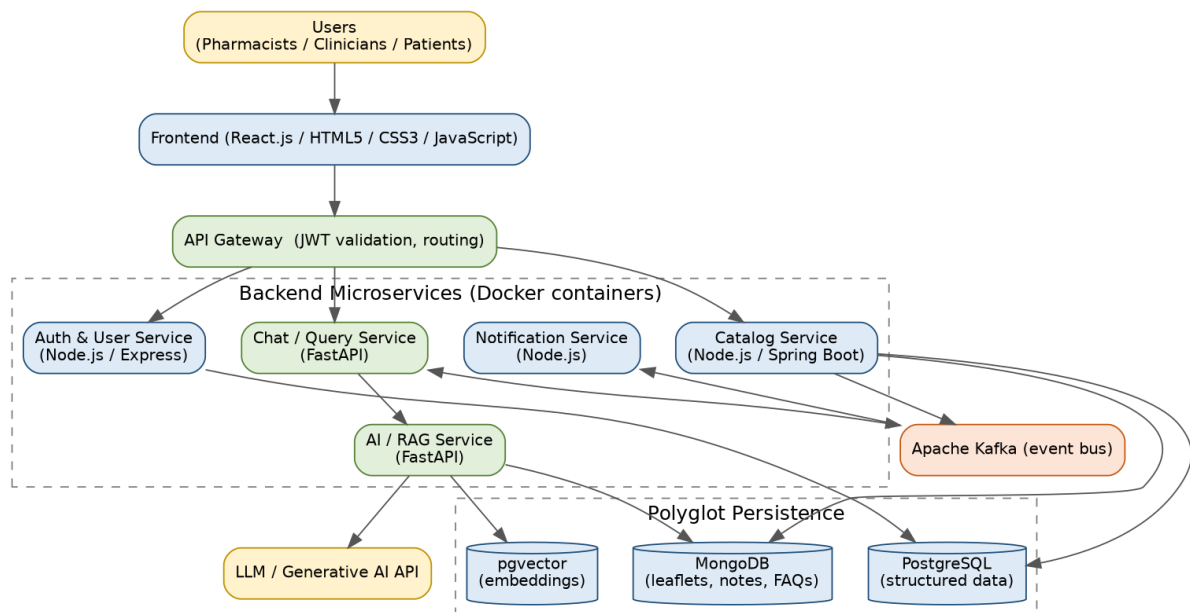


Figure 4.1: High-level architecture of the Generative AI Assistant and Knowledge Base for Pharmaceuticals

4.2 Description of Each Layer

- **Presentation layer (Frontend):** React.js application where users log in, chat with the assistant, view cited sources, search drugs, and (for administrators) manage documents and users.
- **API gateway layer:** single entry point that routes requests to services and validates JWT tokens.
- **Service layer (Backend):** Auth & User Service, Catalog Service, AI/RAG Service, Chat/Query Service and Notification Service. Each is a bounded context that owns its data and can be deployed and scaled independently.
- **Messaging layer:** Apache Kafka carries asynchronous events between services.
- **Data layer:** PostgreSQL (structured), MongoDB (semi-structured) and pgvector (embeddings).
- **AI layer:** embedding model and LLM API used by the RAG pipeline.

RAG request flow

1. The user submits a question from the frontend.
2. The Chat Service authenticates the request and forwards the query to the AI/RAG Service.
3. The query is converted to an embedding and pgvector returns the most similar knowledge chunks.

4. Related structured facts (for example, interactions) are fetched from PostgreSQL, and the full leaflet text from MongoDB when needed.
5. A prompt containing the question and the retrieved context is sent to the LLM with instructions to answer only from that context.
6. The answer and its source references are returned to the user and stored in the chat history.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

Every service is packaged as a Docker image. For local development, Docker Compose starts all services together with PostgreSQL, MongoDB and Kafka using a single command. For cloud deployment the same images are pushed to a container registry and run on a cloud container platform.

The CI/CD pipeline is implemented with GitHub Actions:

1. On every push or pull request, the pipeline installs dependencies and runs linting and unit tests for each service.
2. On merge to the main branch, Docker images are built and pushed to the registry.
3. The deployment stage pulls the new images and restarts the affected services.

The design is documented with the C4 model – System Context, Container, Component and Code views – so that new developers can understand the system at the level of detail they need.

CHAPTER - 5 DESIGN

5.1 C4 Model Views

C4 level	Description
Level 1 – Context	Users (pharmacist, clinician, patient, admin) interact with the Pharma AI Assistant system, which in turn uses an external LLM/embedding API.
Level 2 – Container	Frontend web app, API gateway, Auth, Catalog, AI/RAG, Chat and Notification services, Kafka, PostgreSQL, MongoDB and pgvector.
Level 3 – Component	Inside the AI/RAG service: query handler, embedding client, retriever, prompt builder, LLM client and response formatter.
Level 4 – Code	Key classes and modules such as Pydantic schemas, repository classes and the JWT dependency.

5.2 ER Diagram / Database Schema

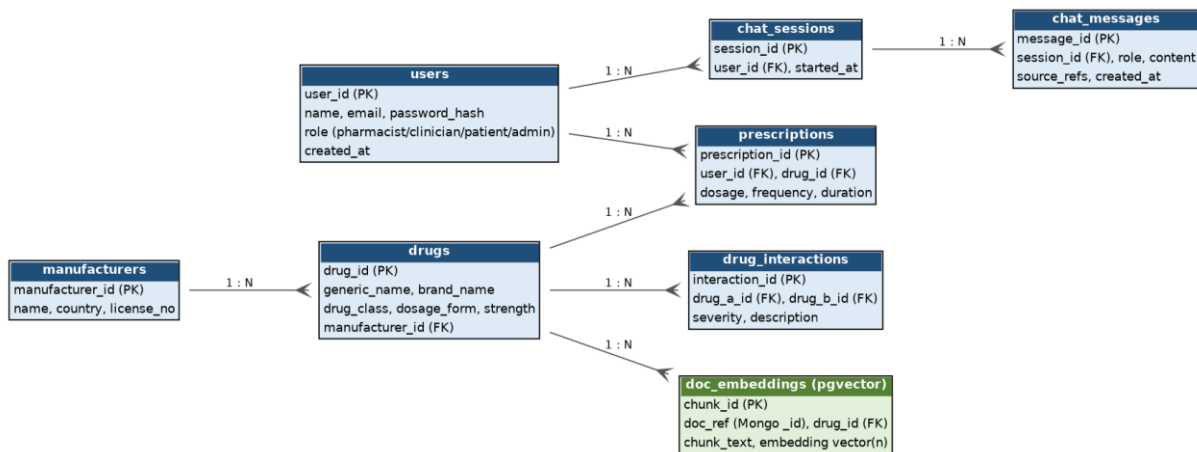


Figure 5.1: Entity-Relationship diagram of the relational schema and vector table

The relational schema is normalised to Third Normal Form. A manufacturer produces many drugs; a drug can take part in many interactions and prescriptions; a user owns many prescriptions and chat sessions; each chat session has many messages. The doc_embeddings table stores the vector representation of each knowledge chunk together with a reference to the source document in MongoDB and, where applicable, the related drug.

Sample SQL (PostgreSQL + pgvector)

```

CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE doc_embeddings (
    chunk_id SERIAL PRIMARY KEY,
    doc_ref TEXT, drug_id INT REFERENCES drugs(drug_id),
    chunk_text TEXT, embedding vector(1536));
SELECT chunk_text FROM doc_embeddings
ORDER BY embedding <=> $1 LIMIT 5;

```

CHAPTER - 6 IMPLEMENTATION

6.1 Module-wise Implementation

- **Authentication & User Module:** registration, login, role management and JWT issuing; passwords are stored as salted hashes.
- **Catalog Module:** CRUD operations for drugs, manufacturers and interactions in PostgreSQL, and leaflet/FAQ management in MongoDB. Publishes events to Kafka when content changes.
- **Knowledge Ingestion Module:** consumes “document added/updated” events, splits documents into chunks, generates embeddings and stores them in pgvector.
- **AI / RAG Module:** performs retrieval, builds the grounded prompt, calls the LLM and returns the answer with sources.
- **Chat / Query Module:** manages chat sessions and message history and coordinates calls to the AI/RAG service.
- **Notification Module:** consumes Kafka events and sends alerts (for example, when a monitored drug record is updated).
- **Admin Module:** user management, document upload and system monitoring.

6.2 Front-End Logic

The React frontend is organised into pages (Login, Chat, Drug Search, Admin) and reusable components (MessageBubble, SourceList, DrugCard, ProtectedRoute). Authentication state is kept in a context provider; the JWT is attached to every API call. The chat component keeps the message list in state, shows a loading indicator while the answer is generated, and renders the returned source references beneath each answer so users can verify the information.

6.3 API Integration

All services expose REST endpoints documented automatically through OpenAPI/Swagger. Representative endpoints are listed below.

Method	Endpoint	Purpose
POST	/auth/register, /auth/login	Create a user; obtain a JWT
GET	/drugs?query=	Search drugs in the catalog
GET	/drugs/{id}/interactions	List known interactions of a drug
POST	/chat/sessions	Start a chat session
POST	/chat/query	Ask the assistant; returns answer + sources
POST	/admin/documents	Upload a leaflet or guideline for ingestion

Services communicate synchronously over REST for request/response operations and asynchronously through Kafka topics for events such as document ingestion and notifications.

6.4 Authentication & Authorization

Authentication is JWT-based. After a successful login the Auth Service issues a signed, time-limited token containing the user id and role. The API gateway and each FastAPI service validate the token through a dependency-injected security function. Role-based access control (RBAC) restricts sensitive operations: only administrators can upload documents or manage users, while pharmacists, clinicians and patients can use the assistant and search the catalog. Pydantic models validate all incoming data, and secrets such as the signing key and API keys are supplied through environment variables, never hard-coded.

CHAPTER - 7 FEATURES

7.1 List of Main Features

1. Natural-language pharmaceutical Q&A powered by a generative AI assistant.
2. Retrieval-augmented answers that cite their sources.
3. Drug search with details on class, form, strength and manufacturer.
4. Drug-interaction lookup.
5. Dosage guideline and regulatory-document queries.
6. Role-based login for pharmacists, clinicians, patients and administrators.
7. Chat history per user.
8. Admin console for uploading and updating knowledge documents.
9. Event-driven notifications.
10. Interactive API documentation (Swagger) and containerised one-command setup.

7.2 How Each Feature Works

Feature	User perspective	Technical description
AI Q&A	User types a question and receives a clear answer with references.	Query is embedded, top-k chunks are retrieved from pgvector, a grounded prompt is sent to the LLM.
Drug search	User searches by generic or brand name and sees a drug card.	Parameterised SQL query on PostgreSQL through the Catalog Service.
Interaction lookup	User selects two drugs and sees severity and description.	Join on the drug_interactions table; result may also be summarised by the LLM.
Leaflet / FAQ access	User opens the leaflet text behind an answer.	Document fetched from MongoDB using the doc_ref stored with the embedding.
Login & roles	User logs in and sees features allowed for their role.	JWT issued by Auth Service; RBAC enforced in the gateway and services.
Chat history	User revisits earlier conversations.	chat_sessions and chat_messages tables in PostgreSQL.
Document ingestion	Admin uploads a new guideline; it becomes searchable shortly afterwards.	Upload event published to Kafka; ingestion service chunks, embeds and stores vectors.
Notifications	User is informed about relevant updates.	Notification Service consumes Kafka events and delivers messages.

CHAPTER - 8 TESTING

8.1 Postman Testing (Frontend/Backend)

Every REST endpoint was tested with Postman before being connected to the frontend. A Postman collection was created with one folder per service. Environment variables hold the base URL and the JWT token, which is captured automatically from the login response and reused in later requests. For each request, the status code, response schema and response time were checked using Postman test scripts. Negative tests (missing token, invalid payload, wrong role) confirmed that the services return the correct 401, 403 and 422 responses.

8.2 Integration Testing

Integration tests verified that services work together: Frontend → Gateway → Chat Service → AI/RAG Service → databases and LLM. Kafka-based flows were tested by uploading a document and confirming that embeddings were created and that the new content appeared in retrieved answers. The whole stack was started with Docker Compose so that tests ran against real PostgreSQL, MongoDB and Kafka containers.

8.3 Tools Used for Testing

Tool	Purpose
Postman	Manual and scripted API testing
Swagger UI	Exploring and trying endpoints from the OpenAPI specification
pytest	Unit tests for FastAPI services
Jest / JUnit	Unit tests for Node.js or Spring Boot services
Docker Compose	Provisioning the integration test environment
GitHub Actions	Running the test suite automatically on every push

8.4 Test Cases and Results

The table lists the planned test cases and expected results. Fill in the Status column after executing the tests.

ID	Test case	Expected result	Actual result	Status
TC01	Register with valid details	201 Created; user stored	[]	[Pass/Fail]
TC02	Login with wrong password	401 Unauthorized	[]	[Pass/Fail]
TC03	Access chat without token	401 Unauthorized	[]	[Pass/Fail]
TC04	Patient calls admin upload endpoint	403 Forbidden	[]	[Pass/Fail]
TC05	Search drug by generic name	List of matching drugs	[]	[Pass/Fail]

ID	Test case	Expected result	Actual result	Status
TC06	Query interaction of two drugs	Severity and description returned	[]	[Pass/Fail]
TC07	Ask a question covered by the knowledge base	Grounded answer with sources	[]	[Pass/Fail]
TC08	Ask a question not covered by the knowledge base	Assistant states information is unavailable instead of guessing	[]	[Pass/Fail]
TC09	Upload a new leaflet	Embeddings created; new content retrievable	[]	[Pass/Fail]
TC10	Invalid request body	422 with validation message	[]	[Pass/Fail]

CHAPTER - 9 DEPLOYMENT

9.1 Steps to Deploy

1. Clone the repository from GitHub.
2. Create a .env file with the required variables (see 9.2).
3. Build the images: docker compose build.
4. Start the complete system: docker compose up -d. This starts PostgreSQL (with pgvector), MongoDB, Kafka, all microservices and the frontend.
5. Run database migrations and the seed script to load initial drug data.
6. Run the ingestion script to embed the initial pharmaceutical documents.
7. Open the frontend in a browser and the Swagger UI of each service to verify.

9.2 Environment Configuration

Variable	Description
POSTGRES_URL	Connection string of PostgreSQL / pgvector
MONGO_URL	Connection string of MongoDB
KAFKA_BROKERS	Address of Kafka brokers
JWT_SECRET, JWT_EXPIRY	Secret and lifetime for JWT tokens
LLM_API_KEY, LLM_MODEL	Credentials and model name for the generative AI API
EMBEDDING_MODEL	Embedding model used for ingestion and queries
CORS_ORIGINS	Allowed frontend origins

Secrets are never committed to Git; in CI/CD they are stored as GitHub Actions secrets.

9.3 Hosting of Frontend and Backend

The frontend can be built into static files and hosted on any static hosting service or served by an Nginx container. Backend microservices run as containers on a cloud container platform, with managed PostgreSQL and MongoDB or self-hosted containers with persistent volumes. GitHub Actions builds and pushes images and triggers redeployment automatically.

CHAPTER - 10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Hallucination: the LLM sometimes answered from its own memory instead of the retrieved context.
- Chunking quality: poorly sized chunks returned incomplete or irrelevant passages.
- Decomposing the monolith: deciding service boundaries and data ownership was difficult.
- Data consistency across PostgreSQL, MongoDB and pgvector when documents changed.
- Service-to-service configuration in Docker Compose (networking, start-up order for Kafka and databases).
- Latency of the LLM API and embedding calls.

10.2 Solutions Applied

- Strict prompt instructions to answer only from the supplied context, with a fallback message when nothing relevant is found; sources are always shown.
- Tuned chunk size and overlap, and stored metadata to improve retrieval.
- Applied domain-driven design – bounded contexts for auth, catalog, retrieval, chat and notification.
- Used Kafka events to keep the vector store synchronised with content changes.
- Added health checks and depends_on conditions in Docker Compose.
- Used asynchronous FastAPI endpoints and limited the number of retrieved chunks to reduce latency.

10.3 Current Limitations or Known Bugs

- Answer quality depends on the coverage and quality of documents in the knowledge base.
- The system depends on external LLM and embedding APIs, so an internet connection and API quota are required.
- Responses may be slower during high load.
- The assistant is not a substitute for professional medical advice and is not connected to real patient records.
- Currently supports English content only.

CHAPTER - 11 FUTURE ENHANCEMENTS

11.1 Planned Features

- Multilingual support so that patients can ask questions in regional languages.
- Voice input and output for accessibility.
- Personalised drug-interaction checks against a patient's prescription list.
- Automatic ingestion of updates from regulatory sources.
- Answer-feedback mechanism (helpful / not helpful) to improve retrieval.
- Mobile application.

11.2 Possible Integrations or Optimizations

- Integration with hospital and pharmacy information systems using healthcare standards such as HL7 FHIR.
- Hybrid search (keyword + vector) and re-ranking to improve retrieval accuracy.
- Response caching and semantic caching to reduce cost and latency.
- Kubernetes deployment with autoscaling and centralised monitoring (logging, metrics, tracing).
- Automated evaluation of answer accuracy and hallucination rate.
- Self-hosted open-source LLM for data privacy in clinical environments.

CHAPTER - 12 CONCLUSION

12.1 Summary of the Project

This project designed and developed a Generative AI Assistant and Knowledge Base for Pharmaceuticals. By combining a relational database, a document store and a vector database with a RAG pipeline, the system answers drug-related questions using verified, updatable knowledge instead of relying only on an LLM's memory. The backend was built as independently deployable microservices communicating through REST and Kafka, containerised with Docker, and supported by CI/CD and C4 documentation.

12.2 What Was Achieved

- A polyglot persistence layer using PostgreSQL, MongoDB and pgvector.
- A working RAG-based assistant that returns answers with source references.
- A microservices backend with JWT authentication and role-based access.
- Event-driven communication using Apache Kafka.
- A containerised system with an automated CI/CD pipeline.
- A live, demonstrable full-stack application with OpenAPI documentation and C4 architecture views.

12.3 Skills Learned During Development

- Building RAG pipelines: embeddings, vector search and prompt engineering.
- Database design and optimisation across relational, document and vector stores.
- Developing REST APIs with FastAPI, Express.js or Spring Boot.
- Microservice design, decomposition of a monolith and event-driven messaging.
- Docker, Docker Compose and GitHub Actions.
- API testing with Postman and documenting with OpenAPI and the C4 model.
- Teamwork, version control with Git and GitHub, and technical documentation.

CHAPTER - 13 REFERENCES

Books

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Brown, S. (2018). The C4 Model for Visualising Software Architecture.

Papers

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS).

Documentation and Tutorials

FastAPI Documentation. <https://fastapi.tiangolo.com>

Pydantic Documentation. <https://docs.pydantic.dev>

PostgreSQL Documentation. <https://www.postgresql.org/docs>

pgvector. <https://github.com/pgvector/pgvector>

MongoDB Documentation. <https://www.mongodb.com/docs>

Apache Kafka Documentation. <https://kafka.apache.org/documentation>

Docker Documentation. <https://docs.docker.com>

GitHub Actions Documentation. <https://docs.github.com/actions>

React Documentation. <https://react.dev>

Express.js. <https://expressjs.com> | Spring Boot. <https://spring.io/projects/spring-boot>

OpenAPI Specification. <https://swagger.io/specification>

Postman Learning Center. <https://learning.postman.com>

MDN Web Docs. <https://developer.mozilla.org>

C4 Model. <https://c4model.com>

CHAPTER - 14 APPENDICES

Installation / Setup Instructions

1. Install Docker Desktop, Git and (optionally) Python 3.10+ and Node.js.
2. Clone the repository and copy .env.example to .env; fill in database settings, JWT secret and LLM API key.
3. Run docker compose up -d --build.
4. Run the seed and ingestion scripts to load drug data and documents.
5. Open the frontend URL in Chrome / Edge / Firefox; open /docs on each FastAPI service for Swagger UI.

User Manual or Guide

Register or log in with your account. On the Chat page, type a question such as “What are the common side effects of paracetamol?” or “Does drug A interact with drug B?”. The assistant answers and lists the source documents it used; select a source to read the original text. Use Drug Search to look up a medicine by name. Administrators can open the Admin page to upload new leaflets or guidelines and to manage users. Remember that the assistant provides information support only – always consult a qualified healthcare professional for medical decisions.

Geo Tag Photos with Guide

